Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 1:
Socket UDP e TCP

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Implementazione TCP: Byte Stream 3
3.2 Implementazione UDP: Datagram Exchange 3
3.3 Istruzioni di Esecuzione 4

4 Risultati **4**
4.1 Analisi del File Descriptor 4
4.2 Analisi Wireshark 4

5 Conclusioni e Sviluppi Futuri **5**

1 Problem Statement

L'obiettivo di questa prima esercitazione (Hands-On 1) è l'acquisizione delle competenze fondamentali nella programmazione di rete tramite l'interfaccia dei **Berkeley Sockets**. La sfida consiste nel progettare e implementare due architetture client-server basilari per analizzare le divergenze operative tra i due protocolli dominanti del livello di trasporto: **TCP** (Transmission Control Protocol) e **UDP** (User Datagram Protocol). Le specifiche richiedono di validare l'affidabilità del flusso di byte nel TCP e la natura orientata ai datagrammi dell'UDP, ispezionando il comportamento del kernel tramite l'analisi dei file descriptor e del traffico di rete.

2 Stato dell'Arte

La comunicazione nei sistemi distribuiti si basa sulla suite di protocolli TCP/IP, dove il livello di trasporto offre diverse astrazioni di servizio.

- **TCP (RFC 793)**: Fornisce un servizio *byte-stream* affidabile. Gestisce la connessione tramite un *Three-Way Handshake*, garantisce l'ordinamento tramite numeri di sequenza e implementa il controllo di flusso e congestione. È ideale per applicazioni dove la perdita di dati è inammissibile (HTTP, SSH, Database).
- **UDP (RFC 768)**: Fornisce un servizio di invio datagrammi *best-effort* senza connessione. Privo di meccanismi di recupero o setup, garantisce latenze minime ed è lo standard per comunicazioni real-time (VoIP, DNS, Online Gaming).

L'interfaccia di programmazione utilizzata è quella dei **POSIX Sockets**, che astrae la risorsa di rete come un **File Descriptor (FD)**, permettendo l'uso di primitive di I/O standard del sistema operativo.

3 Metodologia ed Implementazione

La soluzione è stata articolata in due coppie di programmi C (Client/Server) che utilizzano le librerie `<sys/socket.h>` e `<arpa/inet.h>`. Lo sviluppo ha seguito una metodologia incrementale: configurazione del socket, binding locale e gestione del flusso dati.

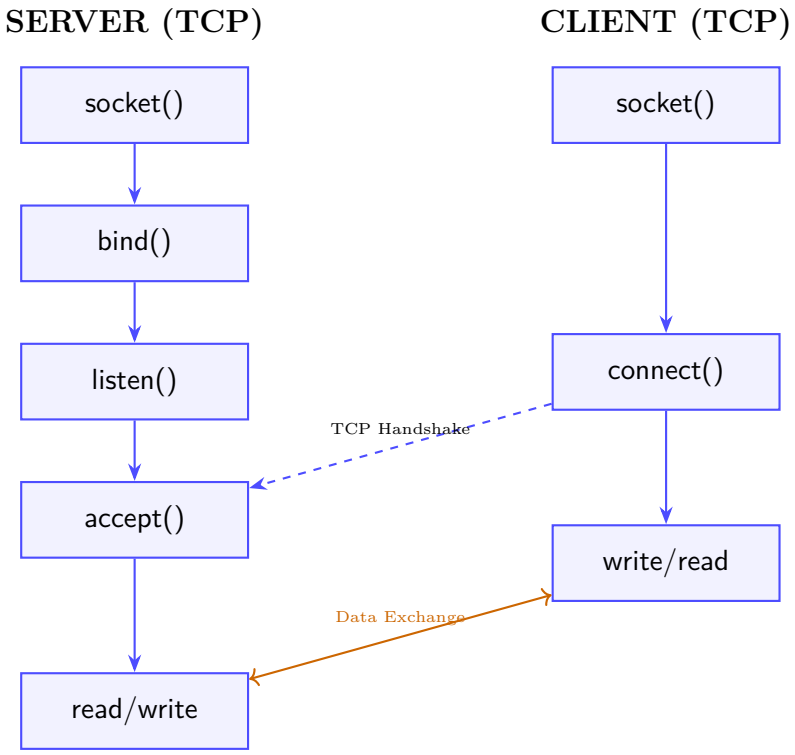

Figure 1: Workflow di interazione tra Client e Server tramite Socket API.

3.1 Implementazione TCP: Byte Stream

Il modello TCP adotta un approccio *stateful*. Il server rimane in ascolto passivo (`listen`) finché non accetta (`accept`) una richiesta di connessione.

Snippet Inizializzazione TCP

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0); // SOCK_STREAM per TCP
bind(server_fd, (struct sockaddr *)&address, sizeof(address));
listen(server_fd, 3);
int new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
```

3.2 Implementazione UDP: Datagram Exchange

Il modello UDP è *stateless*. Viene creato un socket di tipo `SOCK_DGRAM`, permettendo l'invio immediato tramite `sendto` e la ricezione con `recvfrom`, che restituisce contestualmente l'indirizzo del mittente per eventuali repliche.

Snippet Inizializzazione UDP

```
int sockfd = socket(AF_INET, SOCK_DGRAM, 0); // SOCK_DGRAM per UDP
bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr));
recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr *)&cliaddr, &len);
```

3.3 Istruzioni di Esecuzione

Per testare la comunicazione TCP e UDP, compilare i sorgenti con GCC:

Build

```
$ gcc server_tcp.c -o s_tcp
$ gcc client_tcp.c -o c_tcp
$ gcc server_udp.c -o s_udp
$ gcc client_udp.c -o c_udp
```

4 Risultati

La validazione del sistema è stata condotta su due fronti: l'analisi interna delle risorse di sistema e l'ispezione esterna del traffico.

4.1 Analisi del File Descriptor

Esaminando lo pseudo-filesystem `/proc`, è stato confermato che il kernel Linux assegna un intero (`FD 3`) per ogni socket creato, trattandolo come un file aperto. L'utility `lsof` ha validato il binding sulla porta 8080:

Ispezione FD

```
$ ls -l /proc/pid/fd
3 -> socket:[105681]
$ lsof -p pid | grep "UDP"
server_ud  3u  IPv4  UDP *:8080
```

4.2 Analisi Wireshark

La cattura del traffico ha evidenziato graficamente il **Three-Way Handshake** (SYN, SYN-ACK, ACK) nel caso TCP, assente invece nello scambio UDP, confermando le differenze di overhead discusse in teoria.

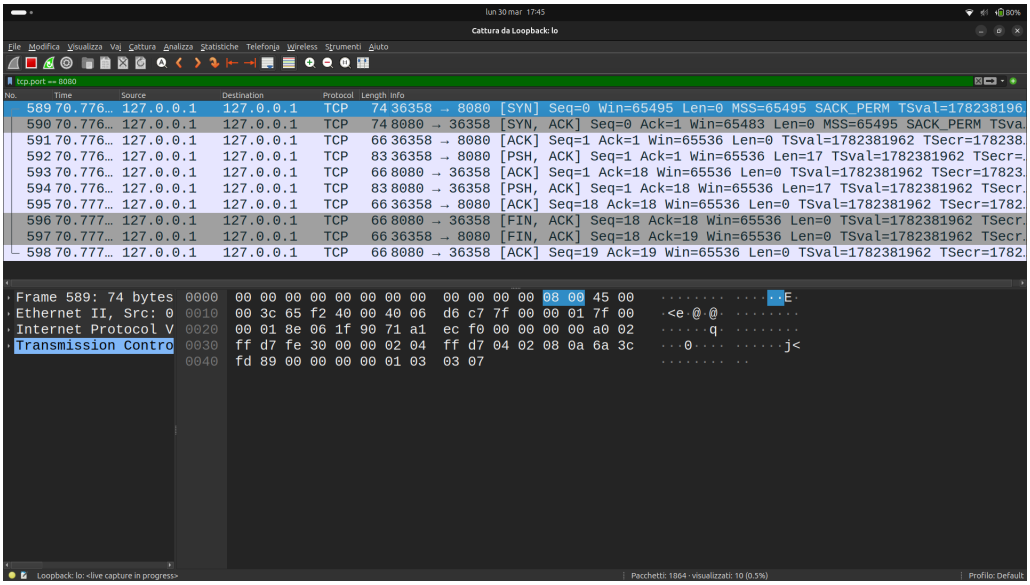

Figure 2: Handshake TCP e trasferimento dati catturati su loopback.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 1 ha permesso di consolidare le basi della programmazione di rete e la comprensione della dualità tra affidabilità (TCP) ed efficienza (UDP). L'integrazione tra codice C e strumenti di diagnostica (Wireshark, lsof) ha fornito una visione *full-stack* della comunicazione di rete.

Sviluppi futuri:

- **Sicurezza**: Introduzione di TLS/SSL per cifrare il canale TCP.
- **Concorrenza**: Evoluzione del server TCP verso un modello basato su `fork()` o thread per gestire connessioni simultanee.
- **Resilienza**: Implementazione di meccanismi di *timeout* e *retry* custom a livello applicativo per il protocollo UDP.